

TP1 IoT - ESP32, DHT11, Node-RED, HTTP REST et MQTT

Ce dépôt regroupe mon travail réalisé dans le cadre du **TP1 d'IoT** consacré à la communication bidirectionnelle entre un **ESP32**, un **capteur DHT11** et une interface **Node-RED**.

Remarque concernant mon environnement de travail

*Dans le cadre de ce TP, j'ai choisi de travailler sur **mon ordinateur personnel**, car je suis plus à l'aise sur cet environnement et j'en maîtrise entièrement la configuration. Cela m'a permis de travailler plus efficacement, de mieux comprendre les éventuels problèmes de compilation ou de dépendances et de garder la maîtrise de l'ensemble de ma chaîne de développement.*

*J'ai utilisé **Visual Studio Code** avec l'extension **PlatformIO**. PlatformIO permet de développer et de téléverser des programmes sur des cartes comme l'ESP32 avec le **framework Arduino**. Le principe reste donc très proche de l'**Arduino IDE** : on écrit le programme, on le compile puis on le téléverse sur la carte. La différence principale est que PlatformIO s'intègre à VS Code et apporte une gestion de projet plus structurée, notamment pour les bibliothèques, les dépendances, les différentes cartes et les paramètres de compilation.*

Je préfère également cette méthode car elle me laisse davantage de contrôle sur le projet. En particulier, il est possible d'ajouter des bibliothèques locales dans le dossier `lib/`, d'utiliser une version précise d'une dépendance ou, si nécessaire, de travailler sur une version modifiée d'une bibliothèque pour l'adapter au besoin du projet.

L'objectif était de mettre en œuvre et de comparer deux méthodes de communication couramment utilisées en IoT :

- **HTTP REST** ;
- **MQTT**, avec un broker **Eclipse Mosquitto**.

Le système doit fonctionner dans les deux sens :

- **Uplink** : l'ESP32 lit la température et l'humidité du DHT11 puis transmet ces mesures vers Node-RED pour les afficher ;
- **Downlink** : une commande envoyée depuis l'interface Node-RED permet de modifier l'état de la LED intégrée à l'ESP32.

Sommaire

- 1. Objectifs du TP
 - 2. Matériel et logiciels utilisés
 - 3. Organisation du dépôt
 - 4. Démarche suivie
 - 5. Lecture du DHT11
 - 6. Communication HTTP REST : POST et GET
 - 7. Node-RED avec REST
 - 8. Communication MQTT
 - 9. Mosquitto et tests MQTT
 - 10. Explication des principales parties du code
 - 11. Comparaison REST et MQTT
 - 12. Lancer le projet
 - 13. Importer le flow Node-RED
 - 14. Corrections et points à vérifier
 - 15. Conclusion
 - 16. Références
-

1. Objectifs du TP

Le sujet demandait de réaliser deux démonstrations de communication bidirectionnelle.

Architecture HTTP REST

L'ESP32 doit :

1. lire la température et l'humidité avec le DHT11 ;
2. envoyer ces informations à Node-RED avec une requête **HTTP POST** ;
3. récupérer l'état demandé pour la LED avec une requête **HTTP GET**.

Node-RED doit :

1. recevoir les données envoyées par l'ESP32 ;
2. afficher les mesures sur un tableau de bord ;
3. proposer une commande permettant d'allumer ou d'éteindre la LED.

Architecture MQTT

L'ESP32 doit :

1. se connecter au broker Mosquitto ;
2. publier la température et l'humidité dans des **topics MQTT** ;
3. s'abonner à un topic de commande de la LED.

Node-RED peut alors publier et souscrire aux mêmes topics via le broker.

2. Matériel et logiciels utilisés

Matériel

- ESP32 **DOIT ESP32 DEVKIT V1** ;
- capteur de température et d'humidité **DHT11** ;
- LED intégrée de l'ESP32, utilisée sur le **GPIO 2** dans mon programme ;
- câbles de connexion ;
- ordinateur connecté au même réseau local que l'ESP32.

Logiciels

- **Visual Studio Code** ;
- **PlatformIO** ;
- framework **Arduino** pour ESP32 ;
- **Node-RED** ;
- **node-red-dashboard 3.6.6** pour l'interface graphique du flow fourni ;
- **Eclipse Mosquitto** comme broker MQTT ;
- bibliothèque **Adafruit DHT sensor library** ;
- bibliothèque **PubSubClient** pour MQTT.

Le paquet historique `node-red-dashboard` utilisé dans ce TP est aujourd'hui déprécié, mais la version 3.6.6 correspond au flow présent dans ce dépôt et reste utile pour reproduire le TP. Pour un nouveau projet de production, il est préférable d'utiliser une solution maintenue comme FlowFuse Dashboard.

3. Organisation du dépôt

J'ai volontairement réalisé le TP dans **plusieurs dossiers/projets PlatformIO indépendants** au lieu de développer directement une seule version finale. Cette organisation correspond aux différentes étapes de mon travail : chaque dossier valide une fonction précise avant de passer à l'étape suivante.

Cela permet notamment :

- d'isoler plus facilement un problème ;
- de vérifier qu'une fonction marche avant d'en ajouter une autre ;
- de conserver les différentes étapes du TP ;
- de comparer facilement les versions REST et MQTT ;

- d'éviter qu'une modification dans une partie du projet casse une étape déjà validée.

Arborescence générale

```
TP1/
|
|-- README.md
|-- CR_IOT_Youssef_EL_KATTOUFI.docx
|
|-- Esp32_DHT_11/
|   |-- platformio.ini
|   |-- src/
|   |   `-- main.cpp
|   |-- include/
|   |-- lib/
|   `-- test/
|
|-- Esp32_to_Node_red/
|   |-- platformio.ini
|   |-- src/
|   |   `-- main.cpp
|   |-- include/
|   |-- lib/
|   `-- test/
|
|-- Esp32_to_Node_red_et_DHT11/
|   |-- platformio.ini
|   |-- src/
|   |   `-- main.cpp
|   |-- include/
|   |-- lib/
|   `-- test/
|
|-- Esp32_to_Mosquitto_et_DHT11/
|   |-- platformio.ini
|   |-- src/
|   |   `-- main.cpp
|   |-- include/
|   |-- lib/
|   `-- test/
|
|-- Node_red_architecture/
|   `-- flow_cpt_hum_temp_dashboard.json
|
|-- DHT-11_datasheet.pdf
|-- TP1_architecture_REST-MQTT_2026.pdf
|-- Tutorial_Getting_Started_ESP32.pdf
```

```
|-- Node_red.url
|-- NodeRED_Manager.bat - Raccourci.lnk
|-- git_push.bat
`-- .gitignore
```

Rôle des dossiers principaux

Dossier	Contenu et rôle
Esp32_DHT_11	Première étape du TP. Ce projet permet de vérifier le câblage du DHT11 , son initialisation et la lecture locale de la température et de l'humidité dans le moniteur série.
Esp32_to_Node_red	Deuxième étape. Ce projet teste uniquement la communication ESP32 -> Node-RED en HTTP POST avec une donnée simple : un compteur. Le but est de valider le Wi-Fi, l'adresse IP du PC, le port 1880, la route HTTP et le fonctionnement de <code>HTTPClient</code> avant d'ajouter le capteur.
Esp32_to_Node_red_et_DHT11	Version REST complète. L'ESP32 lit le DHT11 puis envoie compteur + température + humidité à Node-RED avec un POST /donnees . Dans l'autre sens, l'ESP32 effectue un GET /button afin de récupérer l'état demandé pour la LED.
Esp32_to_Mosquitto_et_DHT11	Version MQTT. L'ESP32 publie la température et l'humidité vers le broker Mosquitto et s'abonne à un topic de commande pour recevoir l'état de la LED.
Node_red_architecture	Contient l'export JSON du flow Node-RED utilisé pour la partie dashboard/REST. Le fichier peut être importé directement dans Node-RED.

Structure d'un projet PlatformIO

Les quatre dossiers ESP32 possèdent globalement la même structure :

```
Nom_du_projet/
|-- platformio.ini
|-- src/
|   |-- main.cpp
|-- include/
|-- lib/
`-- test/
```

- `platformio.ini` : fichier de configuration du projet. Il indique notamment la carte utilisée (`esp32doit-devkit-v1`), le framework (`arduino`), la vitesse du moniteur série et les bibliothèques nécessaires ;
- `src/main.cpp` : fichier contenant le programme principal exécuté par l'ESP32 ;
- `include/` : dossier prévu pour les fichiers d'en-tête personnels (`.h`) ;

- `lib/` : dossier permettant d'ajouter des bibliothèques locales ou personnalisées. C'est notamment utile si je souhaite adapter moi-même une bibliothèque ;
- `test/` : dossier réservé aux tests unitaires PlatformIO ;
- `.pio/` lorsqu'il existe : dossier généré automatiquement par PlatformIO pour les fichiers de compilation et certaines dépendances. Il n'est pas nécessaire de le modifier manuellement.

Autres fichiers du dépôt

- `CR_IOT_Youssef_EL_KATTOUFI.docx` : compte rendu associé au TP ;
- `DHT-11_datasheet.pdf` : documentation du capteur DHT11 ;
- `TP1_architecture_REST-MQTT_2026.pdf` : document de TP / architecture REST-MQTT ;
- `Tutorial_Getting_Started_ESP32.pdf` : documentation de prise en main de l'ESP32 ;
- `flow_cpt_hum_temp_dashboard.json` : export du flow Node-RED ;
- `git_push.bat` : script Windows facilitant l'envoi du projet vers Git ;
- `.gitignore` : liste des fichiers qui ne doivent pas être versionnés.

Cette organisation permet donc de suivre la progression du TP et de comprendre à quel moment chaque fonctionnalité a été ajoutée.

4. Démarche suivie

J'ai volontairement avancé par étapes afin de tester chaque fonction indépendamment.

Étape 1 - Faire fonctionner l'ESP32

J'ai d'abord vérifié :

- la compilation avec PlatformIO ;
- le téléversement sur la carte ;
- le fonctionnement du moniteur série ;
- la LED intégrée.

Étape 2 - Lire le DHT11

J'ai ensuite connecté le DHT11 à l'ESP32 et utilisé la bibliothèque Adafruit DHT afin de récupérer :

- la température en degrés Celsius ;
- l'humidité relative en pourcentage.

Les valeurs ont d'abord été affichées uniquement dans le moniteur série.

Étape 3 - Tester HTTP avec une donnée simple

Avant d'envoyer les mesures du capteur, j'ai testé la communication ESP32 -> Node-RED avec un simple compteur.

Cette étape m'a permis de vérifier séparément :

- la connexion Wi-Fi ;
- l'adresse IP du PC ;
- le port 1880 de Node-RED ;
- la route HTTP ;
- la réception d'une requête POST.

Étape 4 - Ajouter les mesures DHT11 à la requête REST

Une fois le POST validé, j'ai envoyé plusieurs champs dans une seule requête :

```
compteur=42&hum=55.0&temp=23.4
```

Le contenu est envoyé avec le type :

```
application/x-www-form-urlencoded
```

Étape 5 - Ajouter le downlink REST

Pour commander la LED dans l'autre sens, l'ESP32 effectue une requête **GET** vers Node-RED.

Node-RED doit répondre :

```
1
```

pour allumer la LED, ou :

```
0
```

pour l'éteindre.

Étape 6 - Refaire le système avec MQTT

Enfin, j'ai remplacé la logique REST par une architecture MQTT :

- Mosquitto joue le rôle de broker ;
- l'ESP32 publie ses mesures ;
- l'ESP32 s'abonne au topic de commande de la LED ;
- Node-RED peut lui aussi publier et souscrire aux topics.

5. Lecture du DHT11

Le projet correspondant est :

```
Esp32_Dht_11/
```

Dans `platformio.ini`, la bibliothèque DHT est déclarée dans `lib_deps` :

```
lib_deps = adafruit/DHT sensor library@^1.4.7
```

PlatformIO peut donc télécharger automatiquement la dépendance nécessaire lors de la compilation.

Dans le programme :

```
#define DHTPIN 4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
```

Cela signifie que :

- le signal du capteur est relié au **GPIO 4** ;
- le type de capteur est un **DHT11** ;
- l'objet `dht` représente le capteur dans le programme.

Dans `setup()` :

```
dht.begin();
```

initialise le capteur.

Les deux fonctions principales utilisées sont :

```
dht.readTemperature();  
dht.readHumidity();
```

Elles permettent respectivement de récupérer la température et l'humidité.

6. Communication HTTP REST : POST et GET

6.1 Architecture générale

flowchart LR
DHT[DHT11] --> ESP[ESP32]
ESP -- "POST /donnees
compteur + température + humidité" --> NR[Node-RED]
NR -- "Bouton LED" --> ESP
ESP -- "GET /button" --> NR
NR -- "Réponse : 0 ou 1" --> ESP
ESP --> LED[LED interne GPIO 2]

Dans cette partie du TP, l'**ESP32 est un client HTTP** et **Node-RED joue le rôle de serveur HTTP local**.

Les deux méthodes n'ont pas le même rôle :

- le **POST** sert à envoyer des données de l'ESP32 vers Node-RED ;
- le **GET** sert ici à demander à Node-RED l'état que doit prendre la LED.

On obtient donc une communication bidirectionnelle même si, dans les deux cas, c'est l'ESP32 qui initie la requête HTTP.

6.2 Connexion Wi-Fi

Le programme commence par connecter l'ESP32 au même réseau local que l'ordinateur qui exécute Node-RED :

```
WiFi.mode(WIFI_STA);  
WiFi.begin(ssid, password);
```

Le mode `WIFI_STA` signifie que l'ESP32 fonctionne comme un client du réseau Wi-Fi.

Une fois connecté, son adresse IP peut être affichée avec :

```
Serial.println(WiFi.localIP());
```

Pour que les requêtes fonctionnent, l'adresse utilisée dans `serverPOST` et `serverGET` doit correspondre à l'**adresse IP du PC qui héberge Node-RED** sur le réseau local.

6.3 Requête HTTP POST : ESP32 -> Node-RED

Objectif

Le POST est utilisé pour transmettre plusieurs informations mesurées ou générées par l'ESP32 :

- le compteur ;
- l'humidité ;
- la température.

Dans mon programme, la route utilisée est :

```
POST http://IP_DU_PC:1880/donnes
```

Le port 1880 est le port utilisé par défaut par Node-RED.

Création du client HTTP

```
HTTPClient httpPOST;  
httpPOST.begin(serverPOST.c_str());
```

HTTPClient fournit les fonctions nécessaires pour effectuer une requête HTTP depuis l'ESP32.

Type des données envoyées

J'indique ensuite à Node-RED que le corps de la requête est encodé comme un formulaire :

```
httpPOST.addHeader(  
    "Content-Type",  
    "application/x-www-form-urlencoded"  
);
```

Le contenu envoyé est construit sous cette forme :

```
String httpRequestData =  
    "compteur=" + String(compteur) +  
    "&hum=" + String(humidite, 1) +  
    "&temp=" + String(temperature, 1);
```

Exemple réel de corps HTTP :

```
compteur=27&hum=48.0&temp=22.6
```

Le caractère & sépare les différents champs. Node-RED peut ensuite récupérer les valeurs avec :

```
msg.payload.compteur  
msg.payload.hum  
msg.payload.temp
```

Envoi de la requête

La requête est réellement envoyée avec :

```
int httpResponseCode = httpPOST.POST(httpRequestData);
```

La valeur retournée correspond au **code de réponse HTTP**.

Par exemple :

- 200 : requête correctement traitée ;
- 404 : route inexistante ;
- une valeur négative retournée par la bibliothèque ESP32 indique généralement un problème de connexion ou de transport.

Le programme affiche également la réponse de Node-RED :

```
String reponse = httpPOST.getString();  
Serial.println(reponse);
```

Enfin :

```
httpPOST.end();
```

termine la requête et libère les ressources du client HTTP.

Pourquoi avoir d'abord créé `Esp32_to_Node_red` ?

Avant d'envoyer directement les données du DHT11, j'ai créé le projet :

```
Esp32_to_Node_red/
```

Celui-ci envoie uniquement :

```
compteur=...
```

Cette étape permet de valider séparément la communication POST. Si le compteur arrive correctement dans Node-RED, je sais que le Wi-Fi, l'adresse IP, la route et le code HTTP fonctionnent avant d'ajouter le DHT11.

6.4 Requête HTTP GET : Node-RED -> ESP32 par interrogation

Objectif

La deuxième partie de la communication permet de commander la LED intégrée de l'ESP32.

Dans mon implémentation, Node-RED conserve l'état demandé par le bouton du dashboard et l'ESP32 vient régulièrement demander cet état avec une requête GET.

La route prévue est :

```
GET http://IP_DU_PC:1880/button
```

Le principe est le suivant :

```
Utilisateur -> bouton Node-RED -> état mémorisé dans Node-RED
                                     ^
                                     |
ESP32 ----- GET /button --+
ESP32 <----- réponse 0/1 ----
```

Il faut donc bien comprendre que **Node-RED n'ouvre pas directement une connexion vers l'ESP32** dans cette version REST. L'ESP32 interroge périodiquement Node-RED pour connaître l'état demandé.

Envoi du GET

Le client est créé avec :

```
HTTPClient httpGET;
httpGET.begin(serverGET.c_str());
```

Puis la requête est envoyée :

```
int getResponseCode = httpGET.GET();
```

Contrairement au POST, aucune donnée n'est placée dans le corps de la requête ici : l'ESP32 demande simplement la ressource `/button`.

Lecture de la réponse

Si Node-RED répond correctement, le corps de la réponse est lu avec :

```
String etatLED = httpGET.getString();
```

Dans ce TP, la convention retenue est :

```
1 -> LED allumée  
0 -> LED éteinte
```

Le programme applique ensuite l'état reçu :

```
if (etatLED == "1")  
{  
    digitalWrite(LED_PIN, HIGH);  
    Serial.println("LED : ON");  
}  
else if (etatLED == "0")  
{  
    digitalWrite(LED_PIN, LOW);  
    Serial.println("LED : OFF");  
}  
else  
{  
    Serial.println("Etat LED inconnu !");  
}
```

Puis :

```
httpGET.end();
```

ferme la requête.

Fréquence d'interrogation

À la fin de la boucle principale, le programme contient :

```
delay(500);
```

L'ESP32 recommence donc son cycle environ toutes les 500 ms, auquel s'ajoute le temps nécessaire aux lectures et aux communications. Il envoie ses données puis interroge Node-RED pour connaître l'état de la LED.

Cette méthode correspond à une forme de **polling** : le client demande régulièrement si la valeur a changé.

6.5 Résumé d'un cycle REST complet

À chaque passage dans `loop()`, le fonctionnement peut être résumé ainsi :

1. lecture de la température et de l'humidité du DHT11 ;
2. incrémentation du compteur ;
3. vérification de la connexion Wi-Fi ;
4. création d'une requête **POST** /`donnees` ;
5. envoi du compteur, de la température et de l'humidité vers Node-RED ;
6. lecture du code et de la réponse HTTP ;
7. fermeture du client POST ;
8. création d'une requête **GET** /`button` ;
9. récupération de 0 ou 1 depuis Node-RED ;
10. mise à jour de la LED sur le GPIO 2 ;
11. fermeture du client GET ;
12. attente puis répétition du cycle.

Le POST assure donc la partie **ESP32 -> Node-RED**, tandis que le GET permet de réaliser la partie **Node-RED -> ESP32** dans l'architecture REST choisie pour ce TP.

7. Node-RED avec REST

Le flow exporté se trouve dans :

```
Node_red_architecture/flow_cpt_hum_temp_dashboard.json
```

7.1 Partie POST du flow Node-RED

Le fichier JSON fourni contient un nœud **HTTP In** configuré pour recevoir une requête :

```
POST /donnees
```

Quand l'ESP32 envoie par exemple :

```
compteur=27&hum=48.0&temp=22.6
```

Node-RED transforme les champs reçus et permet d'accéder à :

```
msg.payload.compteur  
msg.payload.hum  
msg.payload.temp
```

Trois nœuds `function` séparent ensuite les valeurs.

Température

```
msg.payload = msg.payload.temp;  
return msg;
```

Humidité

```
msg.payload = msg.payload.hum;  
return msg;
```

Compteur

```
msg.payload = msg.payload.compteur;  
return msg;
```

Les trois valeurs sont ensuite envoyées vers des graphiques du dashboard.

Un nœud **HTTP Response** est indispensable pour terminer la requête et renvoyer une réponse à l'ESP32. Sans celui-ci, le client HTTP peut rester en attente jusqu'au timeout.

7.2 Partie GET /button pour la commande de LED

Le programme ESP32 de `Esp32_to_Node_red_et_DHT11` contient également :

```
String serverGET = "http://IP_DU_PC:1880/button";
```

La logique Node-RED correspondante doit fournir une route :

```
GET /button
```

qui renvoie l'état du bouton sous la forme :

```
1
```

ou :

```
0
```

Une architecture Node-RED possible pour cette partie est :

```
[Dashboard Button/Switch]
  |
  v
[mémorisation de l'état]

[HTTP In : GET /button]
  |
  v
[récupération de l'état mémorisé]
  |
  v
[HTTP Response : "0" ou "1"]
```

Ainsi, l'utilisateur modifie l'état depuis le dashboard puis l'ESP32 récupère cet état lors de son prochain GET.

7.3 Flow Node-RED final

Le flow final regroupe les deux démonstrations demandées dans le TP :

- un flux **HTTP REST** avec réception des mesures par `POST /donnees` et récupération de l'état de la LED par `GET /button`;
- un flux **MQTT** utilisant Mosquitto avec les topics `esp32/temperature`, `esp32/humidity` et `esp32/output`.

Le dashboard permet ainsi de visualiser les mesures reçues et de commander la LED avec les deux méthodes de communication.

8. Communication MQTT

Le projet correspondant est :

```
Esp32_to_Mosquitto_et_DHT11/
```

Architecture générale

flowchart LR
DHT[DHT11] --> ESP[ESP32]
ESP -- "esp32/temperature" --> B[Broker Mosquitto]
ESP -- "esp32/humidity" --> B
B --> NR[Node-RED]
NR --> DASH[Dashboard]
DASH -- "commande LED" --> NR
NR -- "esp32/output" --> B
B -- "esp32/output" --> ESP
ESP --> LED[LED GPIO 2]

Contrairement à REST, l'ESP32 et Node-RED ne communiquent pas directement l'un avec l'autre. Ils communiquent avec un **broker MQTT**.

Le broker reçoit les messages publiés puis les transmet aux clients abonnés aux topics correspondants.

Création du client MQTT

Le programme utilise :

```
WiFiClient espClient;  
PubSubClient client(espClient);
```

`WiFiClient` fournit la connexion réseau TCP et `PubSubClient` ajoute le protocole MQTT par-dessus.

Adresse du broker

Le broker est défini avec son adresse IP :

```
const char *mqtt_server = "IP_DU_PC";
```

Puis :

```
client.setServer(mqtt_server, 1883);
```

Le port MQTT non chiffré utilisé pour ce TP est 1883.

Topics utilisés

Topic	Sens	Contenu
esp32/temperature	ESP32 -> broker	température du DHT11
esp32/humidity	ESP32 -> broker	humidité du DHT11
esp32/output	broker -> ESP32	commande <code>true</code> ou <code>false</code> pour la LED

Publication des mesures

Les mesures DHT11 sont des nombres flottants. Le code utilise `dtostrf()` pour les convertir en tableaux de caractères avant publication :

```
char tempString[8];
dtostrf(temperature, 1, 2, tempString);
client.publish("esp32/temperature", tempString);
```

Même principe pour l'humidité :

```
char humString[8];
dtostrf(humidite, 1, 2, humString);
client.publish("esp32/humidity", humString);
```

Temporisation avec `millis()`

Le code utilise :

```
long now = millis();
if (now - lastMsg > 500)
```

`millis()` retourne le nombre de millisecondes écoulées depuis le démarrage de l'ESP32.

Cette méthode permet de savoir quand envoyer une nouvelle mesure sans se baser uniquement sur une temporisation bloquante.

Abonnement au topic de commande

Après connexion au broker :

```
client.subscribe("esp32/output");
```

L'ESP32 demande ainsi à recevoir les messages publiés sur ce topic.

La fonction :

```
client.loop();
```

est importante car elle entretient la connexion MQTT et permet de traiter les messages entrants.

Fonction `callback()`

La fonction de callback est appelée lorsqu'un message MQTT correspondant à un abonnement arrive.

Elle récupère :

- le nom du topic ;
- le tableau d'octets contenant le message ;
- la longueur du message.

Le programme reconstruit le texte reçu :

```
String messageTemp;

for (int i = 0; i < length; i++)
{
    messageTemp += (char)message[i];
}
```

Puis vérifie le topic :

```
if (String(topic) == "esp32/output")
```

et commande la LED :

```
if (messageTemp == "true")
{
    digitalWrite(LED_PIN, HIGH);
}
else if (messageTemp == "false")
{
    digitalWrite(LED_PIN, LOW);
}
```

Reconnexion MQTT

La fonction `reconnect()` vérifie que l'ESP32 est connecté au broker.

En cas de perte de connexion, elle tente une reconnexion :

```
if (client.connect("ESP32Client_Youssef"))
```

Puis elle se réabonne au topic de commande :

```
client.subscribe("esp32/output");
```

C'est important, car après une reconnexion le client doit retrouver les abonnements nécessaires au fonctionnement du système.

9. Mosquitto et tests MQTT

Mosquitto est le broker MQTT utilisé localement sur le PC.

Sous Windows, j'ai lancé le broker depuis son dossier d'installation avec une configuration dédiée :

```
cd /d "C:\Program Files\mosquitto"
mosquitto.exe -c "C:\Program Files\mosquitto\mosquitto.conf" -v
```

L'option `-v` affiche les connexions et messages dans le terminal, ce qui est très pratique pour le débogage.

Vérifier les publications de l'ESP32

Depuis un second terminal :

```
mosquitto_sub.exe -h localhost -p 1883 -t "esp32/#" -v
```

Le caractère `#` signifie que l'on souhaite recevoir tous les sous-topics sous `esp32/`.

On doit par exemple voir :

```
esp32/temperature 22.50
esp32/humidity 51.00
```

Tester la LED sans Node-RED

On peut également publier manuellement une commande :

```
mosquitto_pub.exe -h localhost -p 1883 -t "esp32/output" -m "true"
```

Puis :

```
mosquitto_pub.exe -h localhost -p 1883 -t "esp32/output" -m "false"
```

Cela permet de vérifier séparément la partie MQTT de l'ESP32 avant d'ajouter Node-RED.

Le broker doit être accessible depuis l'adresse IP du PC sur le réseau local. Une configuration Mosquitto ou le pare-feu Windows peut empêcher les connexions provenant de l'ESP32. Ne pas exposer un broker anonyme sur Internet.

10. Explication des principales parties du code

`setup()`

La fonction `setup()` est exécutée une seule fois au démarrage ou après un reset.

Dans ce TP, elle sert notamment à :

- démarrer le port série ;
- initialiser le DHT11 ;
- configurer la LED en sortie ;
- se connecter au Wi-Fi ;
- initialiser le client MQTT dans la version MQTT.

`loop()`

La fonction `loop()` est exécutée en permanence.

Elle permet de :

- lire le capteur ;
- envoyer les données ;
- récupérer une commande ;
- maintenir la connexion MQTT ;
- recommencer le cycle.

Serial.begin(...)

Exemple :

```
Serial.begin(9600);
```

Cette instruction initialise la liaison série avec le PC.

La valeur choisie doit correspondre au `monitor_speed` configuré dans `platformio.ini`.

pinMode()

```
pinMode(LED_PIN, OUTPUT);
```

configure la broche comme une sortie numérique.

digitalWrite()

```
digitalWrite(LED_PIN, HIGH);
```

met la sortie à l'état haut.

```
digitalWrite(LED_PIN, LOW);
```

la met à l'état bas.

#define MON_TELEPHONE **et** #ifdef

J'ai utilisé la compilation conditionnelle pour pouvoir basculer entre plusieurs configurations réseau sans réécrire tout le programme.

Principe :

```
#define MON_TELEPHONE
// #define MA_FREEBOX
```

puis :

```
#ifndef MON_TELEPHONE
// configuration 1
#endif

#ifndef MA_FREEBOX
// configuration 2
#endif
```

Une seule configuration doit être activée à la fois.

Pour un dépôt GitHub public, les mots de passe ne doivent cependant pas être écrits directement dans ces blocs.

WiFi.setHostname()

```
WiFi.setHostname(name_card_elec);
```

permet de donner un nom à la carte sur le réseau local.

HTTPClient

Cette classe est utilisée dans l'architecture REST :

```
HTTPClient httpPOST;
```

Puis :

```
httpPOST.begin(serverPOST.c_str());
```

associe le client à l'URL du serveur Node-RED.

WiFiClient + PubSubClient

Dans MQTT :

```
WiFiClient espClient;  
PubSubClient client(espClient);
```

Le premier gère la communication réseau, le second fournit les fonctions MQTT telles que :

```
client.connect(...);  
client.publish(...);  
client.subscribe(...);  
client.loop();
```

11. Comparaison REST et MQTT

Critère	HTTP REST	MQTT
Modèle	Requête / réponse	Publication / abonnement
Intermédiaire	Node-RED agit directement comme serveur HTTP	Broker Mosquitto
Envoi ESP32 -> serveur	POST	<code>publish()</code>
Commande serveur -> ESP32	GET périodique dans cette implémentation	message envoyé directement au topic abonné
Adressage	URL / endpoint	topic
Mise en œuvre	Très intuitive pour une API web	Demande un broker et une logique publish/subscribe
IoT avec beaucoup de messages	Possible, mais plus verbeux	Généralement très adapté
Réactivité du downlink	Dépend du rythme des GET	Très naturelle avec l'abonnement
Débogage	codes HTTP, navigateur, Postman, logs	<code>mosquitto_sub</code> , logs du broker, topics

Ce que j'ai retenu

REST est simple à comprendre car chaque échange est associé à une URL et à une méthode HTTP. Il fonctionne très bien pour communiquer avec une API.

MQTT est particulièrement intéressant en IoT parce qu'un équipement peut publier une information sans connaître directement tous ses destinataires. Le broker se charge de redistribuer les messages aux clients abonnés.

Pour le downlink de ce TP, MQTT est également plus naturel : l'ESP32 reçoit le message dès qu'il est publié, alors que ma version REST interroge régulièrement Node-RED avec un GET.

12. Lancer le projet

Prérequis

Installer :

1. Visual Studio Code ;
2. l'extension PlatformIO ;
3. Node.js et Node-RED ;
4. Mosquitto pour la partie MQTT.

Node-RED

Installation globale avec npm :

```
npm install -g node-red
```

Pour reproduire exactement le dashboard du flow fourni :

```
cd %env:USERPROFILE%\node-red  
npm install node-red-dashboard@3.6.6
```

Démarrer Node-RED :

```
node-red
```

L'éditeur est ensuite accessible localement sur :

```
http://127.0.0.1:1880
```

Avec le dashboard historique, l'interface est généralement accessible sur :

```
http://127.0.0.1:1880/ui
```

PlatformIO

Chaque dossier ESP32 est un projet PlatformIO indépendant.

Il faut donc ouvrir dans VS Code le dossier que l'on souhaite compiler, par exemple :

```
Esp32_to_Mosquitto_et_DHT11
```

PlatformIO utilise le fichier `platformio.ini` pour connaître :

- la plateforme ;
- la carte ;
- le framework ;
- la vitesse du moniteur série ;
- les bibliothèques nécessaires.

Exemple :

```
[env:esp32doit-devkit-v1]
platform = espressif32
board = esp32doit-devkit-v1
framework = arduino
monitor_speed = 9600
lib_deps =
    adafruit/DHT sensor library@^1.4.7
    https://github.com/knolleary/pubsubclient
```

Avec PlatformIO CLI, les commandes équivalentes sont :

```
pio run
pio run -t upload
pio device monitor
```

Outils de lancement fournis

Pour simplifier l'utilisation du projet sous Windows, deux lanceurs sont présents à la racine du TP.

`git_push.bat`

Ce script permet de réaliser rapidement les principales opérations Git nécessaires pour enregistrer les modifications et les envoyer vers le dépôt GitHub. Il exécute successivement :

```
git status
git add .
git commit -m "Maj"
git push
```

Il permet donc de mettre à jour le dépôt sans avoir à retaper manuellement chaque commande dans un terminal.

Lanceur Node-RED

Un second lanceur Windows est fourni pour démarrer plus facilement l'environnement **Node-RED** depuis un menu. Il permet d'accéder rapidement aux commandes nécessaires au lancement de Node-RED sans avoir à les saisir manuellement à chaque utilisation.

Dans le dossier du TP, ce lanceur est accessible via le raccourci :

```
NodeRED_Manager.bat - Raccourci.lnk
```

Ces deux outils ne sont pas nécessaires au fonctionnement du programme ESP32 lui-même, mais ils facilitent l'utilisation quotidienne du projet : l'un pour la gestion du dépôt GitHub et l'autre pour le lancement de Node-RED.

13. Importer le flow Node-RED

Le flow fourni est :

```
Node_red_architecture/flow_cpt_hum_temp_dashboard.json
```

Dans Node-RED :

1. ouvrir l'éditeur ;
2. cliquer sur le menu en haut à droite ;
3. choisir **Import** ;
4. choisir le fichier JSON ou copier son contenu ;
5. importer le flow ;
6. cliquer sur **Deploy**.

Si les nœuds `ui_chart`, `ui_group` ou `ui_tab` sont inconnus, installer le paquet :

```
cd $env:USERPROFILE\.node-red  
npm install node-red-dashboard@3.6.6
```

puis redémarrer Node-RED.

14. Corrections et points à vérifier

En relisant le projet complet, j'ai identifié le point suivant à vérifier dans la version REST.

1. Température et humidité inversées dans la version REST

Dans `Esp32_to_Node_red_et_DHT11/src/main.cpp`, la version actuelle contient :

```
float humidite = dht.readTemperature();  
float temperature = dht.readHumidity();
```

Les noms des variables sont inversés.

La version correcte est :

```
float temperature = dht.readTemperature();  
float humidite = dht.readHumidity();
```

Sinon, le graphique nommé température affiche en réalité l'humidité, et inversement.

15. Conclusion

Ce TP m'a permis de mettre en œuvre une chaîne IoT complète allant d'un capteur physique jusqu'à une interface web.

J'ai commencé par valider séparément chaque élément :

1. lecture du DHT11 ;
2. connexion Wi-Fi ;
3. communication HTTP simple ;

4. transmission de plusieurs mesures vers Node-RED ;
5. commande bidirectionnelle avec REST ;
6. mise en place d'un broker Mosquitto ;
7. publication et abonnement MQTT ;
8. commande de la LED à distance.

Cette progression m'a permis de mieux comprendre la différence entre une architecture **requête/réponse** avec REST et une architecture **publish/subscribe** avec MQTT.

Pour un système IoT simple, REST est facile à mettre en œuvre et à déboguer. MQTT apporte cependant une architecture particulièrement adaptée aux équipements connectés lorsque plusieurs clients doivent échanger des données de manière asynchrone via des topics.

16. Références

- Sujet du TP : TP1_architecture_REST-MQTT_2026.pdf
- Documentation DHT11 fournie : DHT-11_datasheet.pdf
- Tutoriel ESP32 fourni : Tutorial_Getting_Started_ESP32.pdf
- PlatformIO - configuration platformio.ini : <https://docs.platformio.org/en/stable/projectconf/index.html>
(<https://docs.platformio.org/en/stable/projectconf/index.html>)
- PlatformIO - lib_deps :
https://docs.platformio.org/en/latest/projectconf/sections/env/options/library/lib_deps.html
(https://docs.platformio.org/en/latest/projectconf/sections/env/options/library/lib_deps.html)
- Node-RED : <https://nodered.org/> (<https://nodered.org/>)
- Node-RED Dashboard 3.6.6 : <https://flows.nodered.org/node/node-red-dashboard>
(<https://flows.nodered.org/node/node-red-dashboard>)
- Eclipse Mosquitto : <https://mosquitto.org/> (<https://mosquitto.org/>)
- Mosquitto mosquitto_sub : https://mosquitto.org/man/mosquitto_sub-1.html
(https://mosquitto.org/man/mosquitto_sub-1.html)
- PubSubClient : <https://github.com/knolleary/pubsubclient> (<https://github.com/knolleary/pubsubclient>)
- GitHub - ajouter un projet local : <https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github>
(<https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github>)

Auteur

TP réalisé dans le cadre d'un enseignement d'IoT / **systèmes connectés**.

